

City Bike & Scooter Availability Explorer

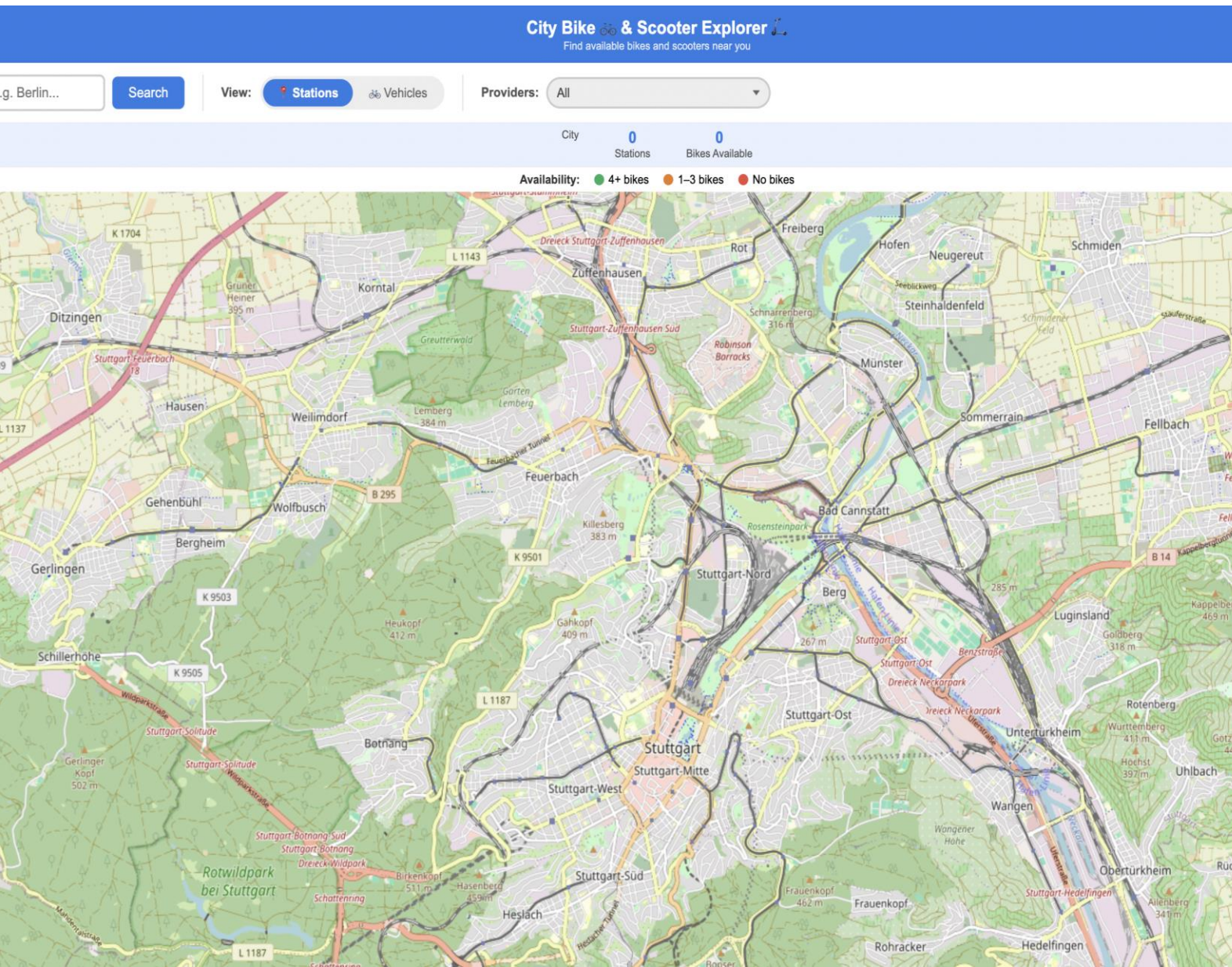
1. **Pranavi Mukkasa** (100008165)
2. **Ashutosh Londhe** (100008419)



Problem Statement

- Finding available bikes in a city is difficult
- Everyday thousands of people: Walk to a bike station only to find it empty
- 📍 Users do not know which stations have bikes before going there
- 🌐 No single platform covers multiple bike operators in one city

Solution Overview

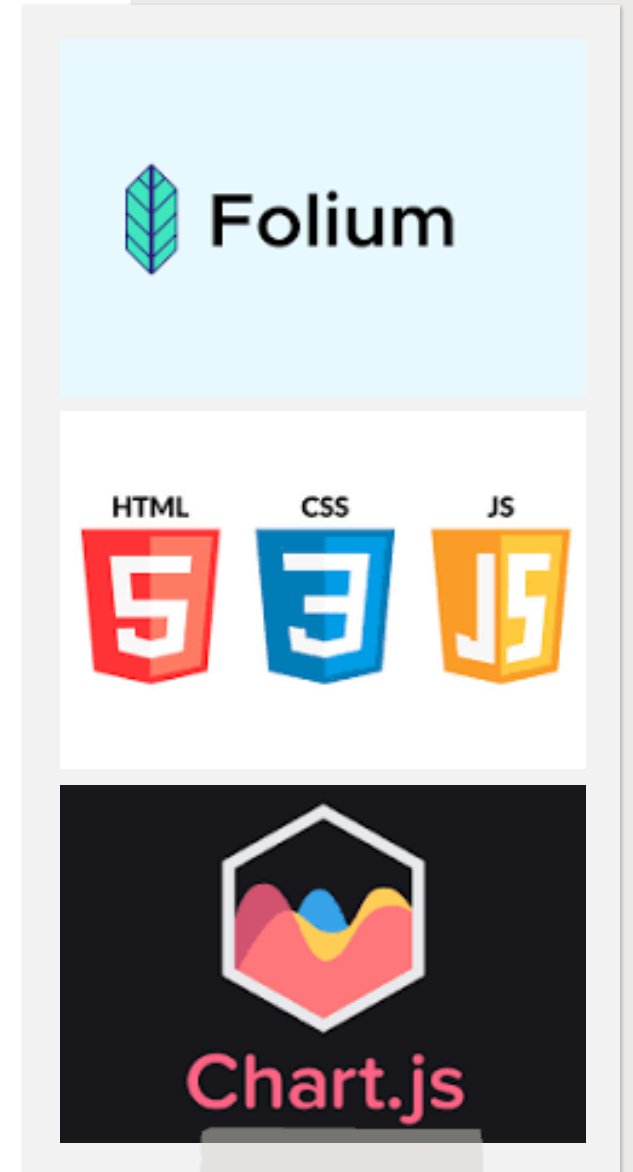


City Bike & Scooter Availability Explorer

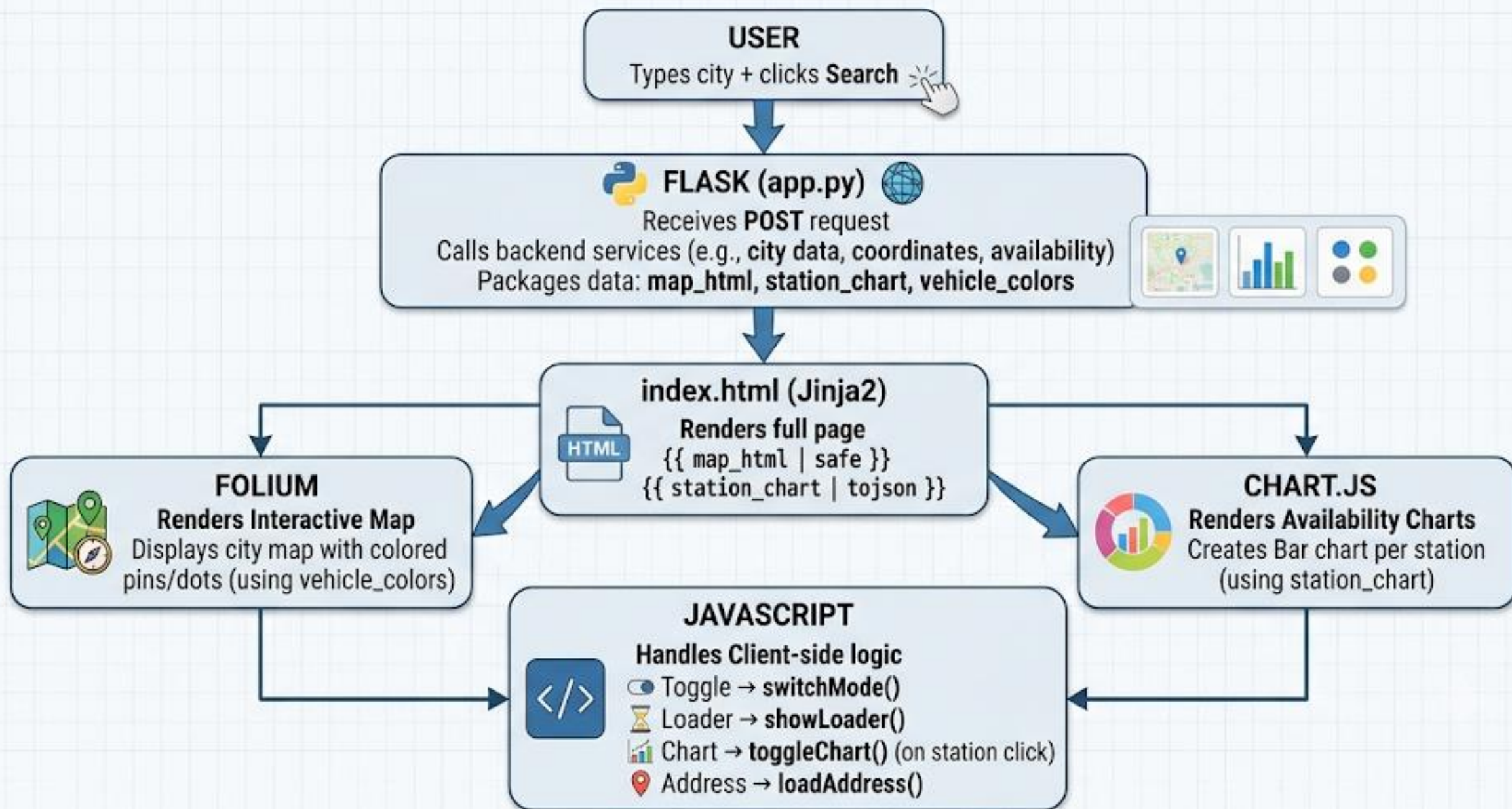
1. Search any city worldwide → 1500+ cities supported via GBFS
2. See all stations on an interactive map → Color coded by availability
3. A dropdown telling which providers are available
4. View individual vehicles → Each dot = one real bike or scooter
5. See vehicle specifications → Range, speed, type on one click

Front end Tech Stack

- **Folium** : Python library that builds interactive maps
- **Chart.js** : JavaScript library loaded from CDN
- **HTML + CSS + JavaScript** : Search bar and toggle switch and the layout of the web application , JavaScript handled all user interactions in the browser after page loads
- **Flask + Jinja2** : Connects Python data to HTML
- **Example** - `{{ map_html | safe }}` renders the map
- **Nominatim API**: Free OpenStreetMap reverse geocoding
 - Converts station coordinates to street address
 - Called only when user clicks the button

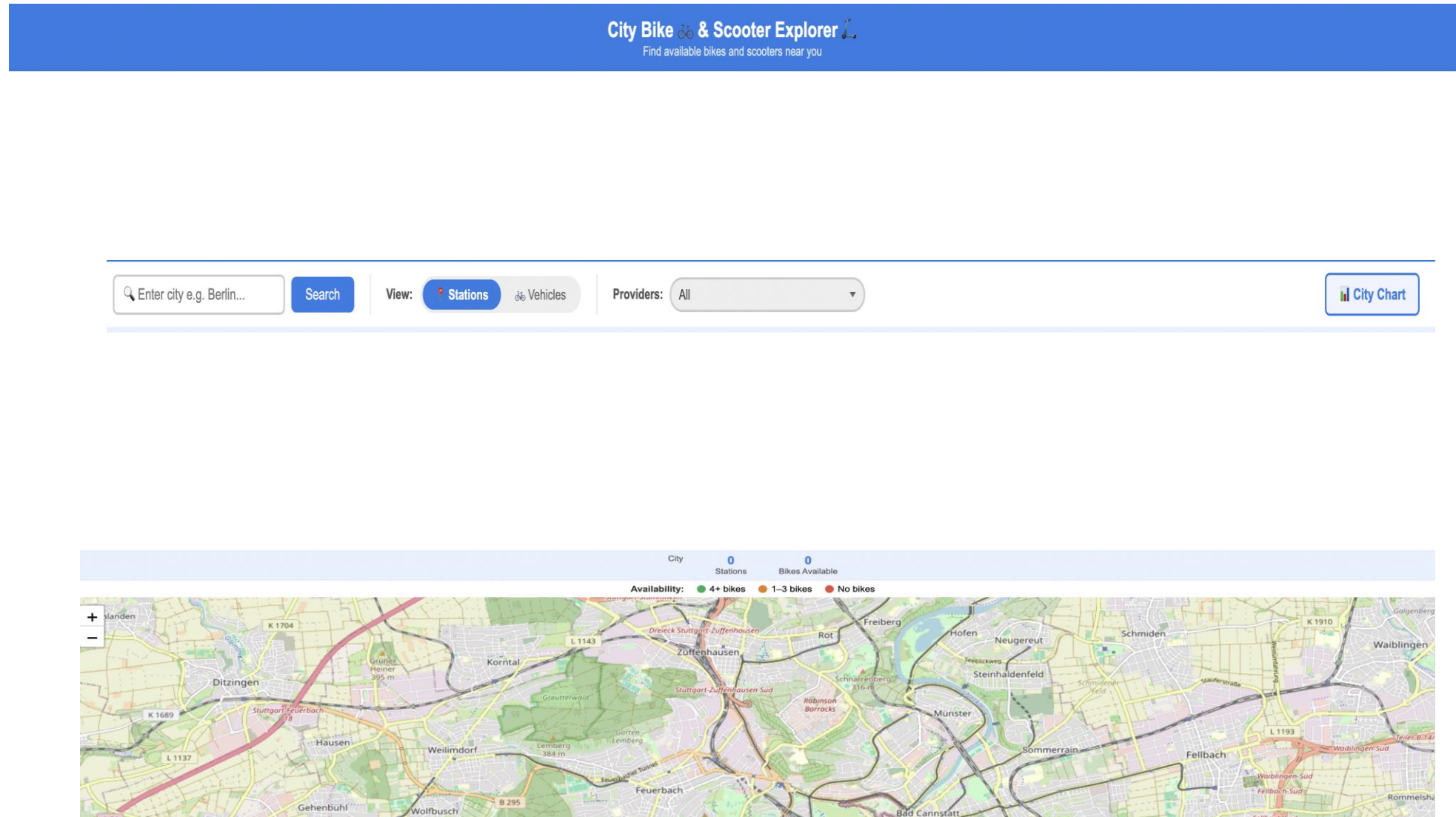


FRONTEND ARCHITECTURE FLOW CHART



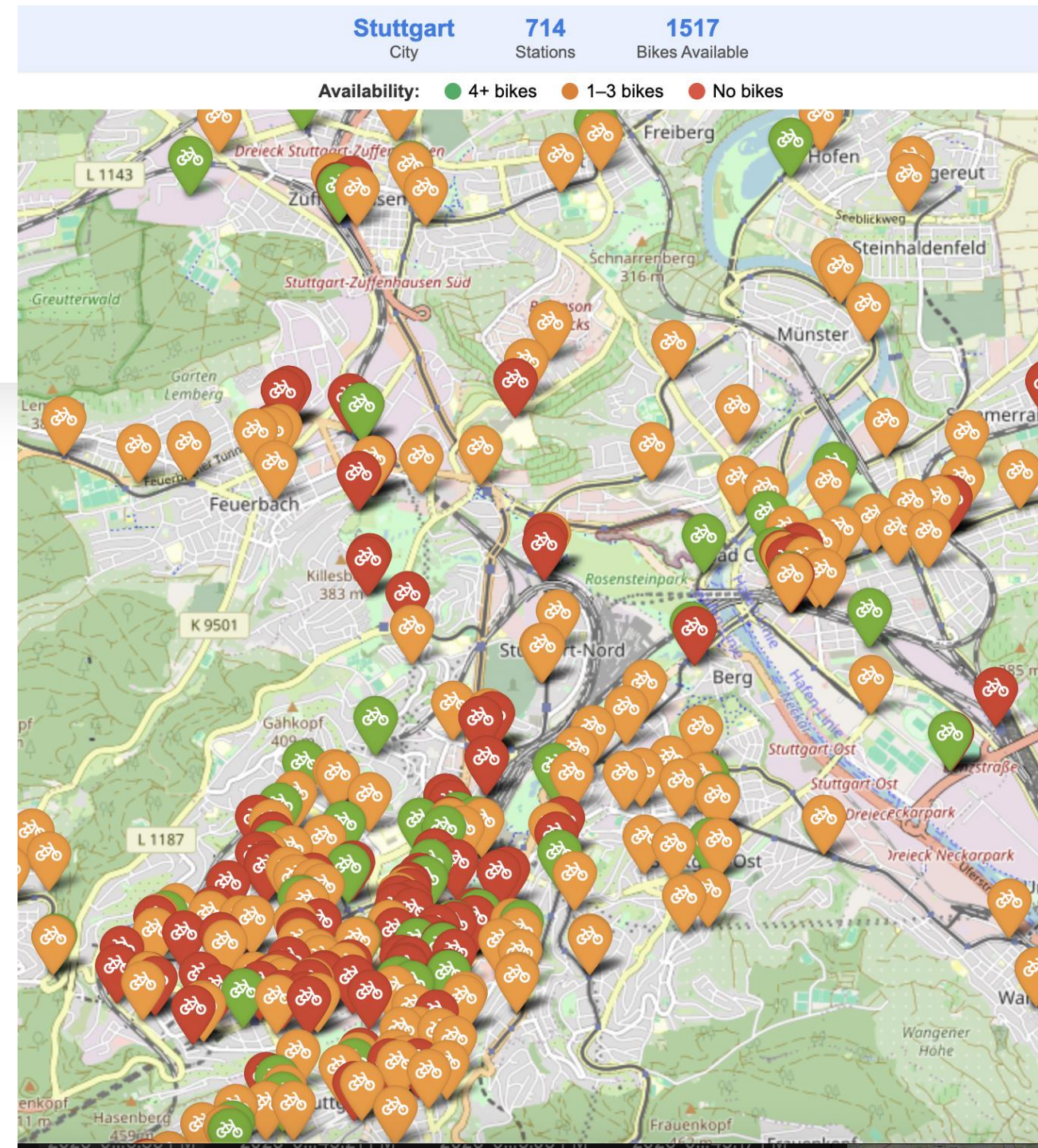
What does the user see ?

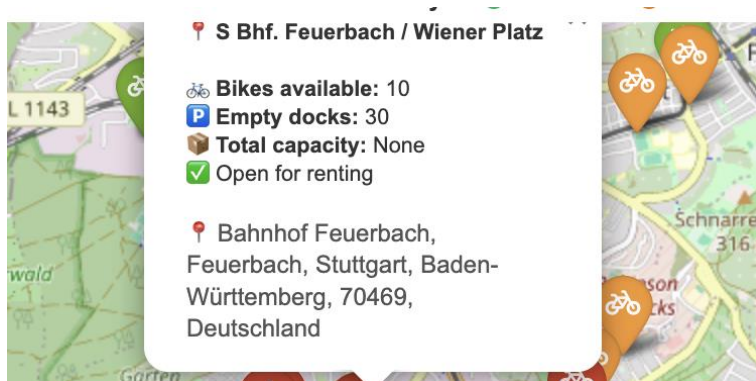
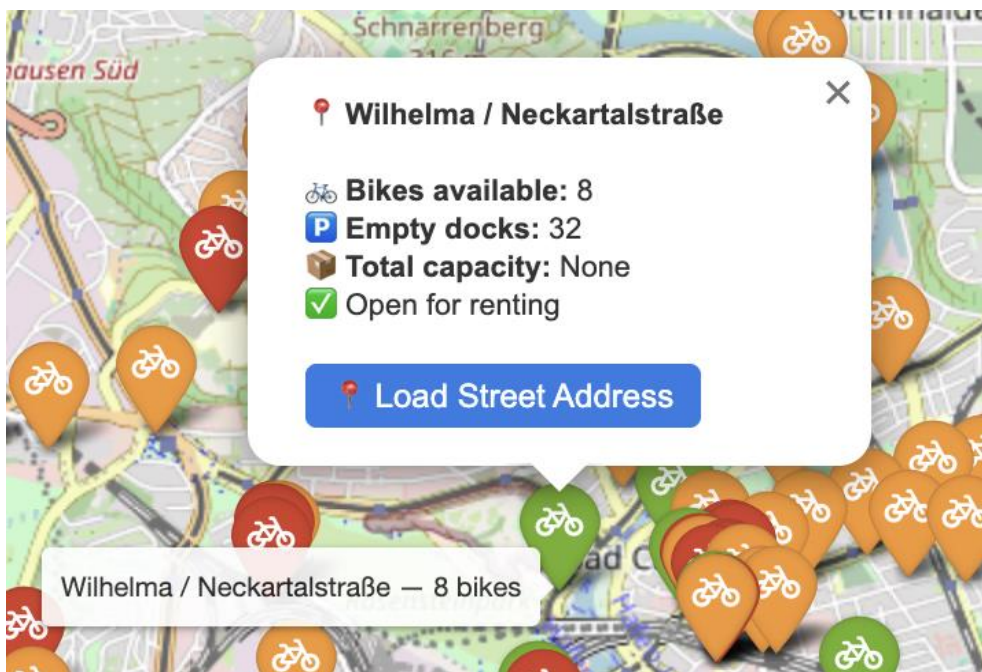
- **Header Bar**
- **Search Bar**
- **Filter Dropdown for Providers**
- **Stations/Vehicles Toggle**
- **City Chart Button**
- **Live Stats Bar**
- **Legends guide for number of vehicles and type of vehicles**
- **Interactive Map**
- **Ranked Chart Panel**



Stations View Demo

- Stations Mode - Live Bike Availability
What each color means:
- Green = 4 or more bikes ready to rent
- Orange = 1 to 3 bikes, running low
- Red = No bikes
- Map is locked to searched city: →
Cannot drag to another city → Cannot
zoom out past city level →
Automatically fits all stations on
screen



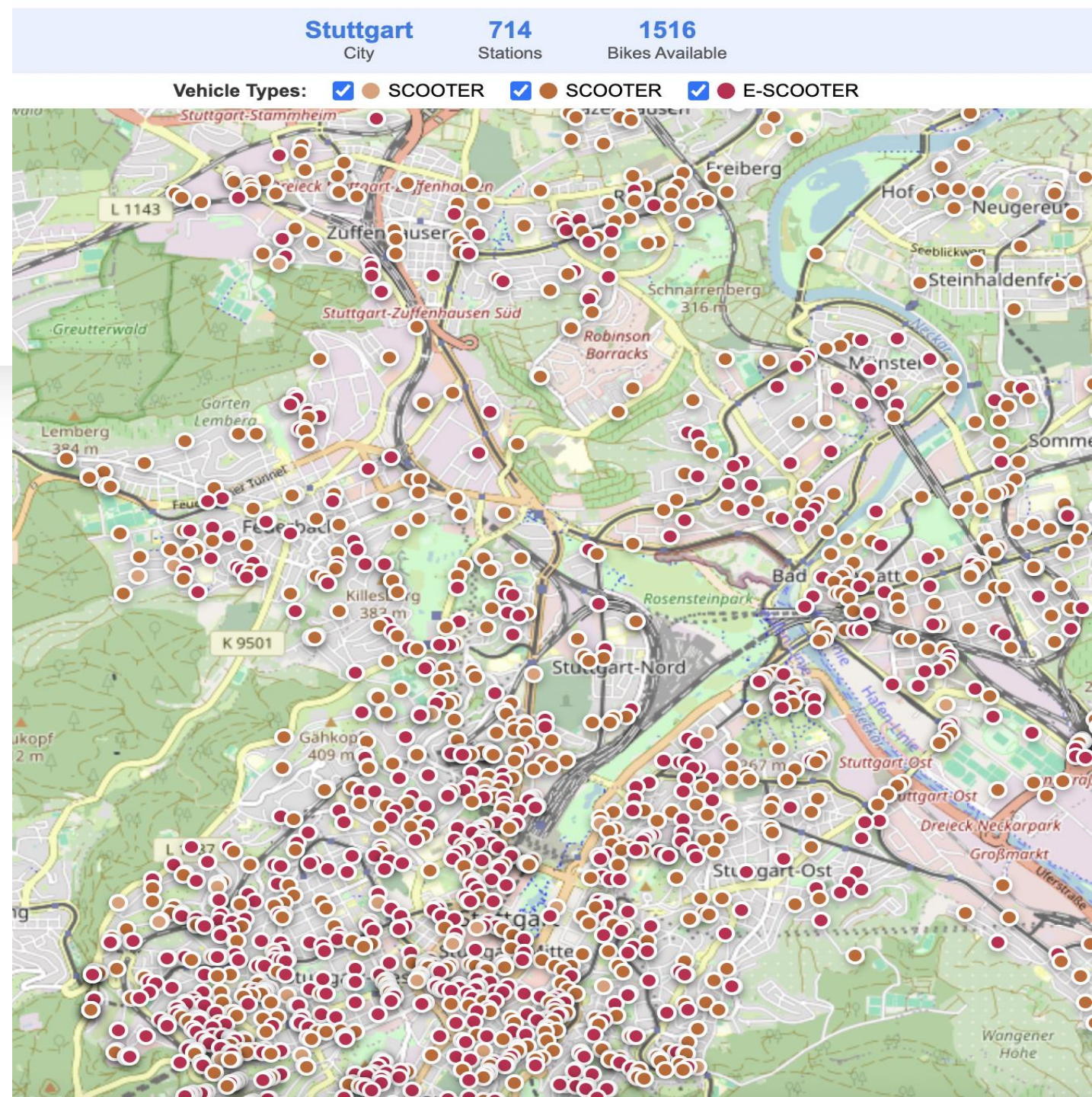


Station Pop up

-  Station Name
 - Exact name from GBFS feed
-  Bikes Available
 - Live count right now
-  Empty Docks
 - How many spots to return a bike
-  Total Capacity
 - Maximum bikes the station holds
-  /  Open for Renting
 - Whether station is currently active
-  Load Street Address Button
 - Calls Nominatim API
 - Returns full address like: "Hauptstraße 12, Mitte, Berlin"
 - Only loads when clicked (saves API calls)

Vehicles View Demo

- Vehicles Mode -Individual Vehicle Dots
- What you see:
 - → Each dot = one real bike or scooter parked at that exact GPS location
 - → Color = vehicle type → Dozens of dots clustered at stations



Vehicles Pop Up

Click any vehicle dot to see:



Form Factor

→ bicycle / scooter / cargo_bicycle



Propulsion Type

→ electric_assist / human / electric



Maximum Range

→ How far on one full charge

- Example: 70 km



Maximum Speed

→ Speed limit for this vehicle

- Example: 25 km/h



Wheel Count

→ Number of wheels on the vehicle



Rider Capacity

→ How many people can use it



Street Address

→ Exact location of this vehicle

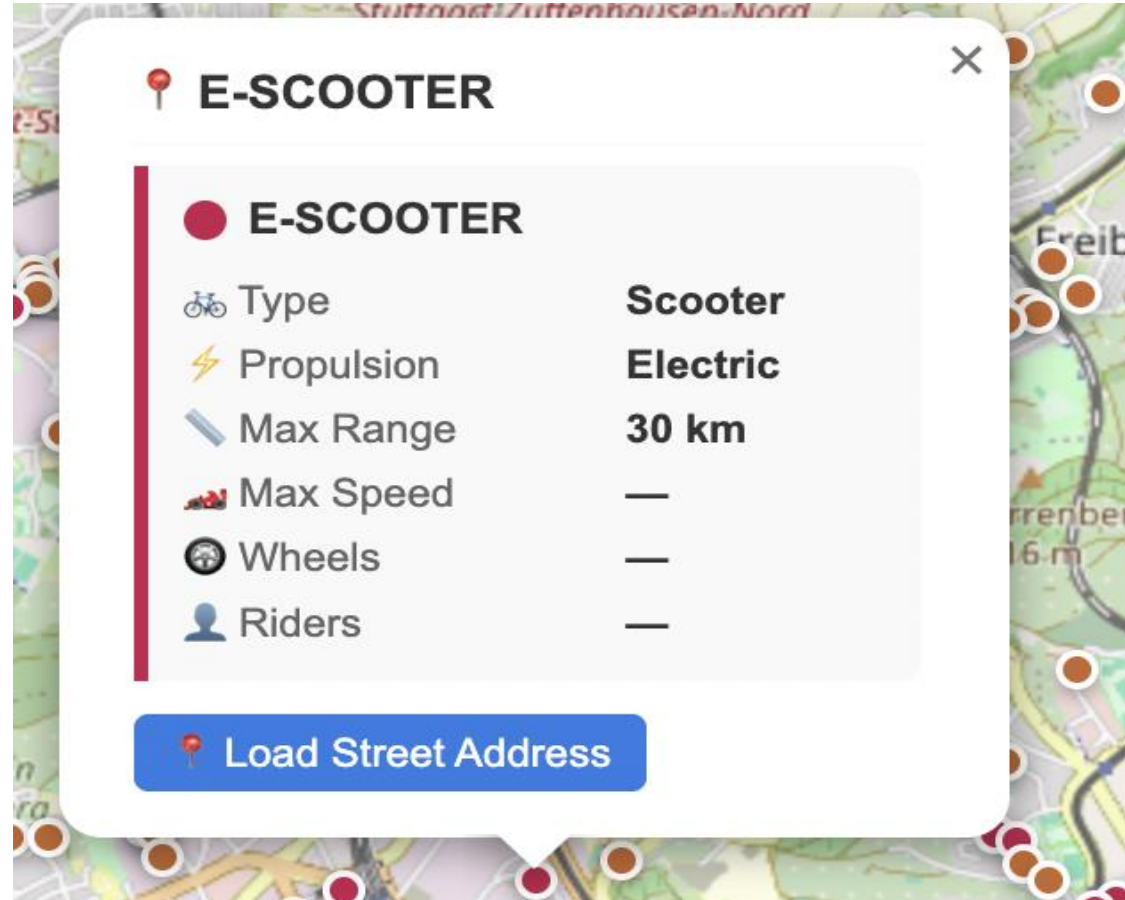
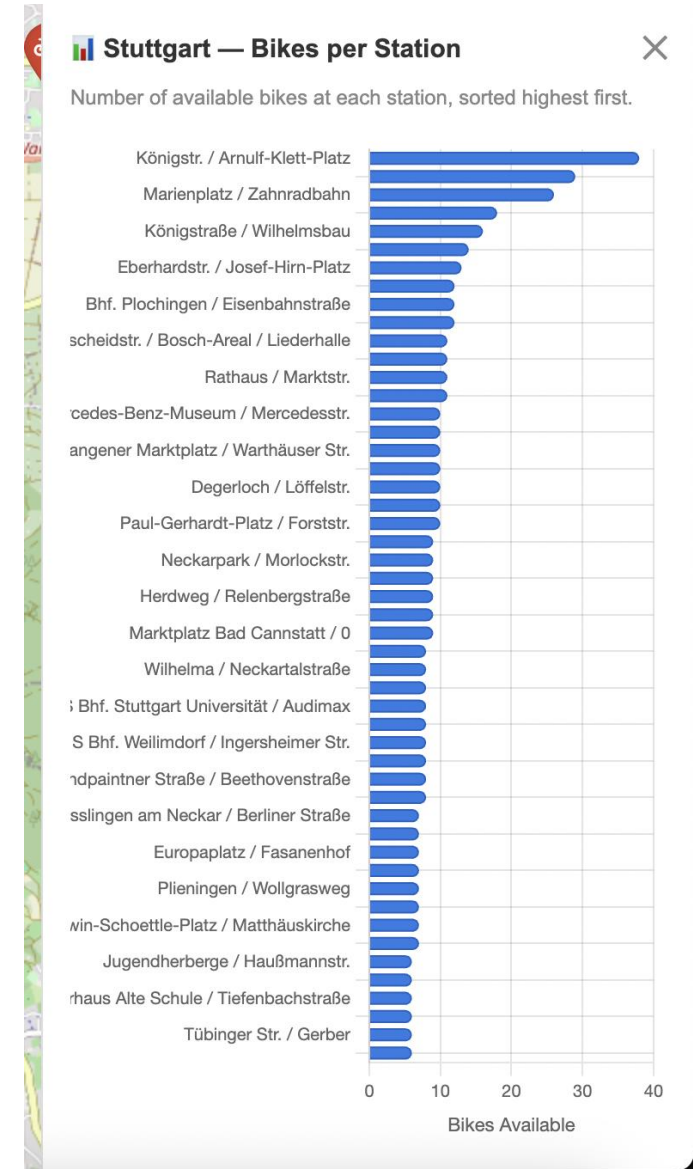


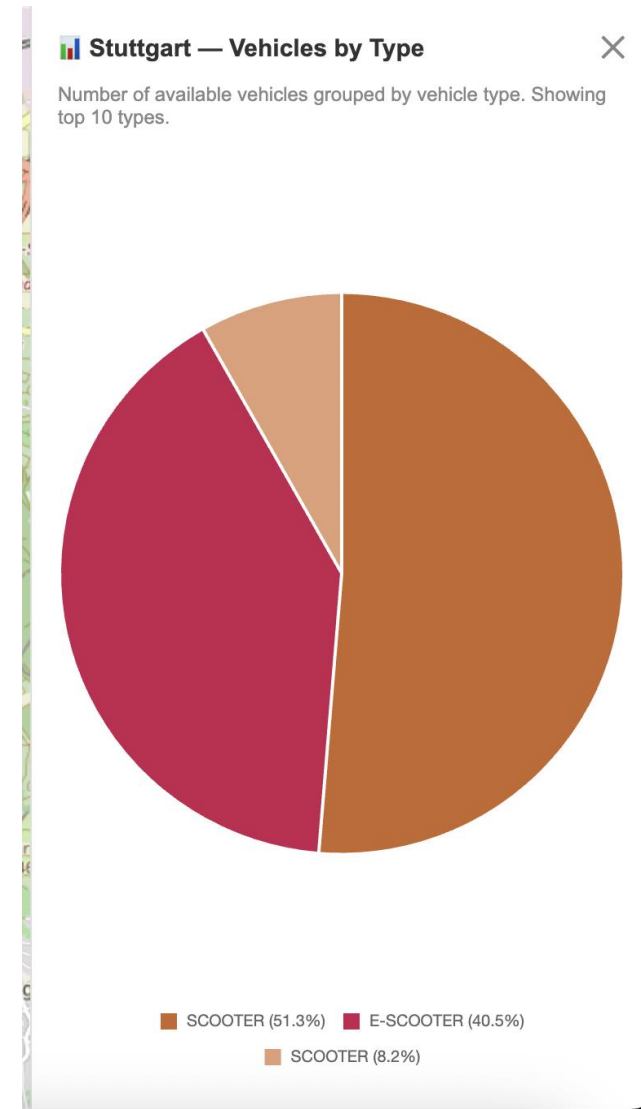
Chart Visualization

1) Bar Chart - City Chart - Bikes Per Station How to open: → Click 🗺️ City Chart button when toggle is on Stations view

- → Panel slides in smoothly from right side →
- What it shows: → Every station in the searched city
- → Station names on the left (Y axis)
- → Number of bikes on the right (X axis)
- → Sorted from most bikes at top to least bikes at bottom



- 2) Pie Chart - City Chart – Vehicles by Type
- How to open: → Click 📊 City Chart button when toggle is on Vehicles view
- → Panel slides in smoothly from right side →
- What it shows: → Every vehicle in the searched city
- → Along with how many vehicles of that type are available



FrontEnd Test Cases

1)Empty Search Field

What was tested:

- Clicking Search button without typing any city name

Input:

- User leaves search bar empty and clicks Search button

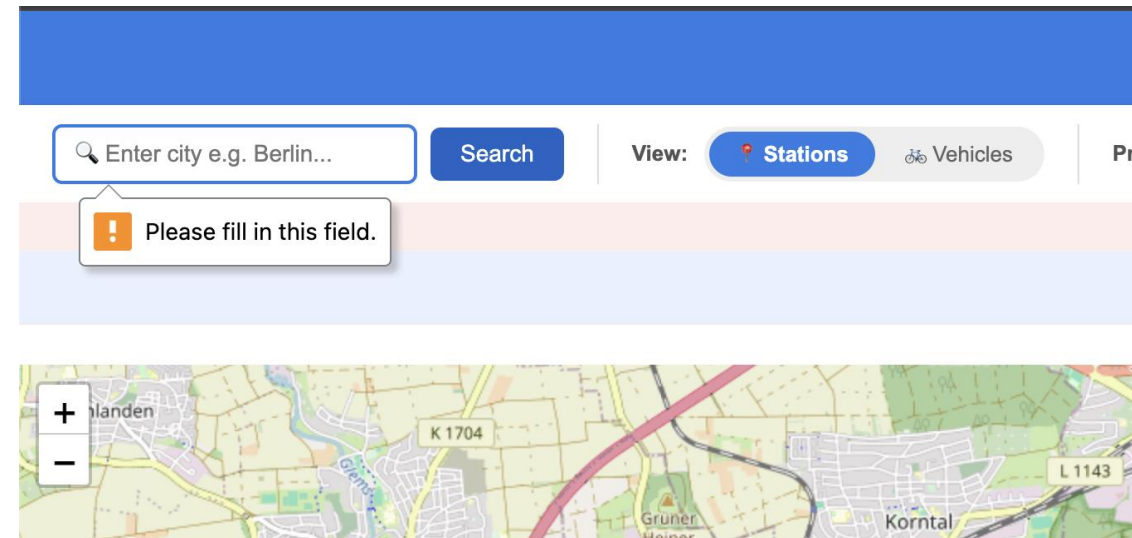
Expected Result:

- Browser prevents form submission and prompts user to fill in the field

Actual Result:

- Browser shows validation popup: "Please fill in this field."
Form does not submit ,Map stays unchanged No API call is made

Result:  **PASS**



2)Invalid City Search

What was tested:

- Searching for a city that does not exist
- in the GBFS systems list

Input:

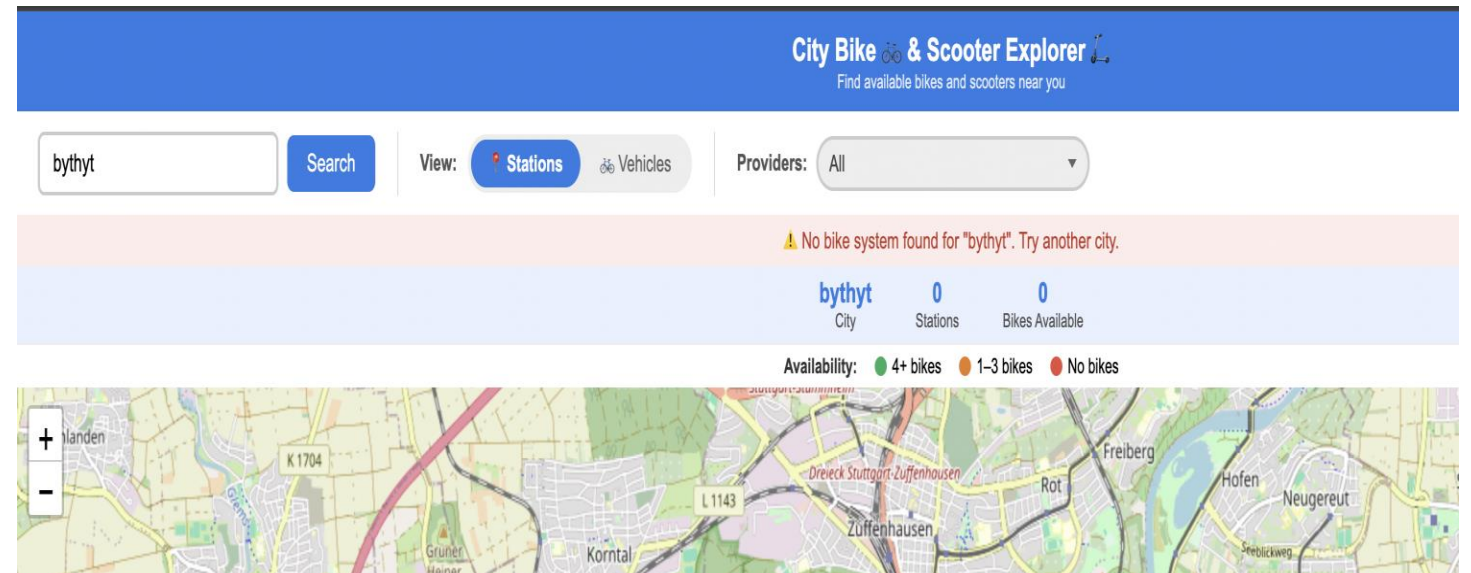
- User types "bythyt" and clicks Search

Expected Result:

- Error message appears telling the user
- the city was not found

Actual Result:

- "No bike system found for bythyt. Try another city."
- Stats show 0 Stations, 0 Bikes Available
- Map stays on previous view
-



3) Bar Chart Without Searching a City

What was tested:

- Opening the City Chart panel before searching for any city

Input:

- User clicks City Chart button without searching a city first

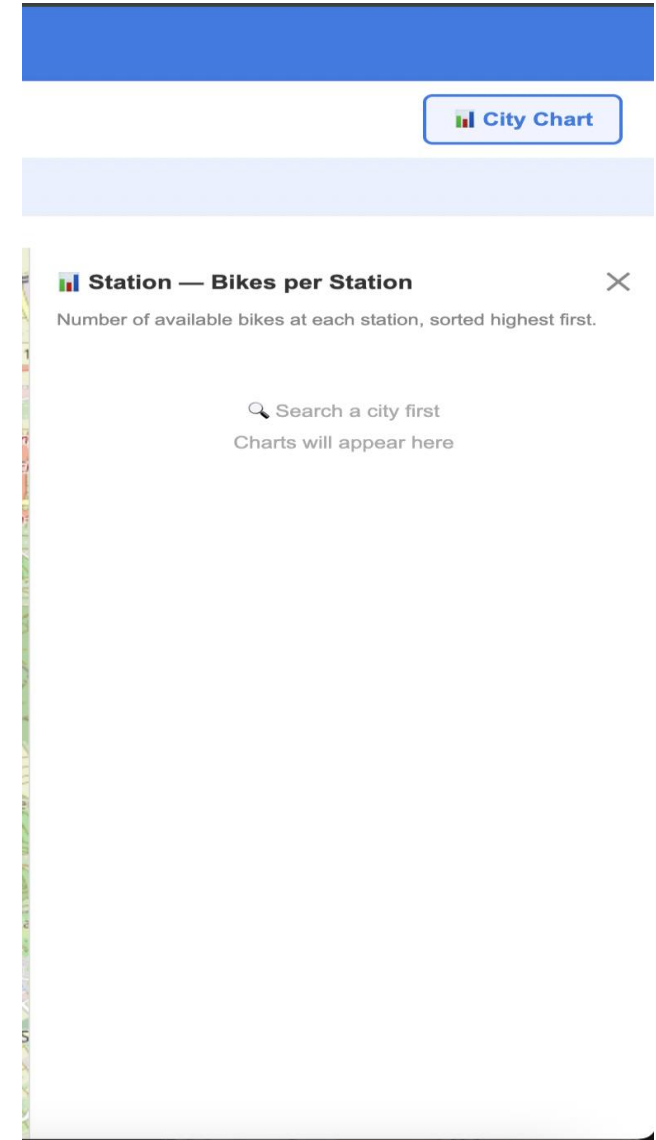
Expected Result:

- Chart panel opens but shows a helpful message instead of empty chart

Actual Result:

- Chart panel slides in correctly Shows: "Search a city first"
"Charts will appear here", No broken chart or error shown

Result:  **PASS**



4)Map Bounds Locked To City

What was tested:

- Attempting to drag the map outside the boundaries of the searched city
Input:
- Step 1: User searches "Stuttgart"
- Step 2: Map loads with Stuttgart stations
- Step 3: User tries to drag map toward another city (e.g. Munich)

Expected Result:

- Map should stay within Stuttgart User should not be able to see other cities on the map

Actual Result:

- Map automatically fits to Stuttgart when search loads When user tries to drag outside: → Map bounces back to Stuttgart → Grey empty area visible at edges → Cannot reach any other city → Zoom out is also restricted

Result:  **PASS**

- **Demonstrated in Live**

FrontEnd Challenges

1)Nominatim API Rate Limit

- Nominatim allows only 1 request per second - loading addresses for all stations at once would freeze the app for minutes.
- Solution: Added a "Load Street Address" button so the address only fetches when the user actually wants it for that station.

2)Too Many Providers On Map

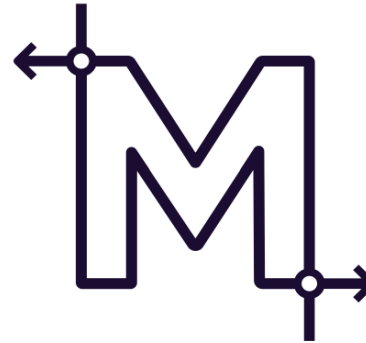
- Cities like Berlin have multiple bike providers (Nextbike, Bolt, Jelbi) making the map cluttered with hundreds of overlapping pins.
- Solution: Added a Providers dropdown so the user can filter and see only one provider's stations at a time.

3)No Visual Feedback During Data Fetch

- When the user clicked Search the page went blank for 5-8 seconds with no indication of whether the app was working or crashed.
- Solution: Added a loader spinner that appears immediately on Search so the user knows data is being fetched in the background.



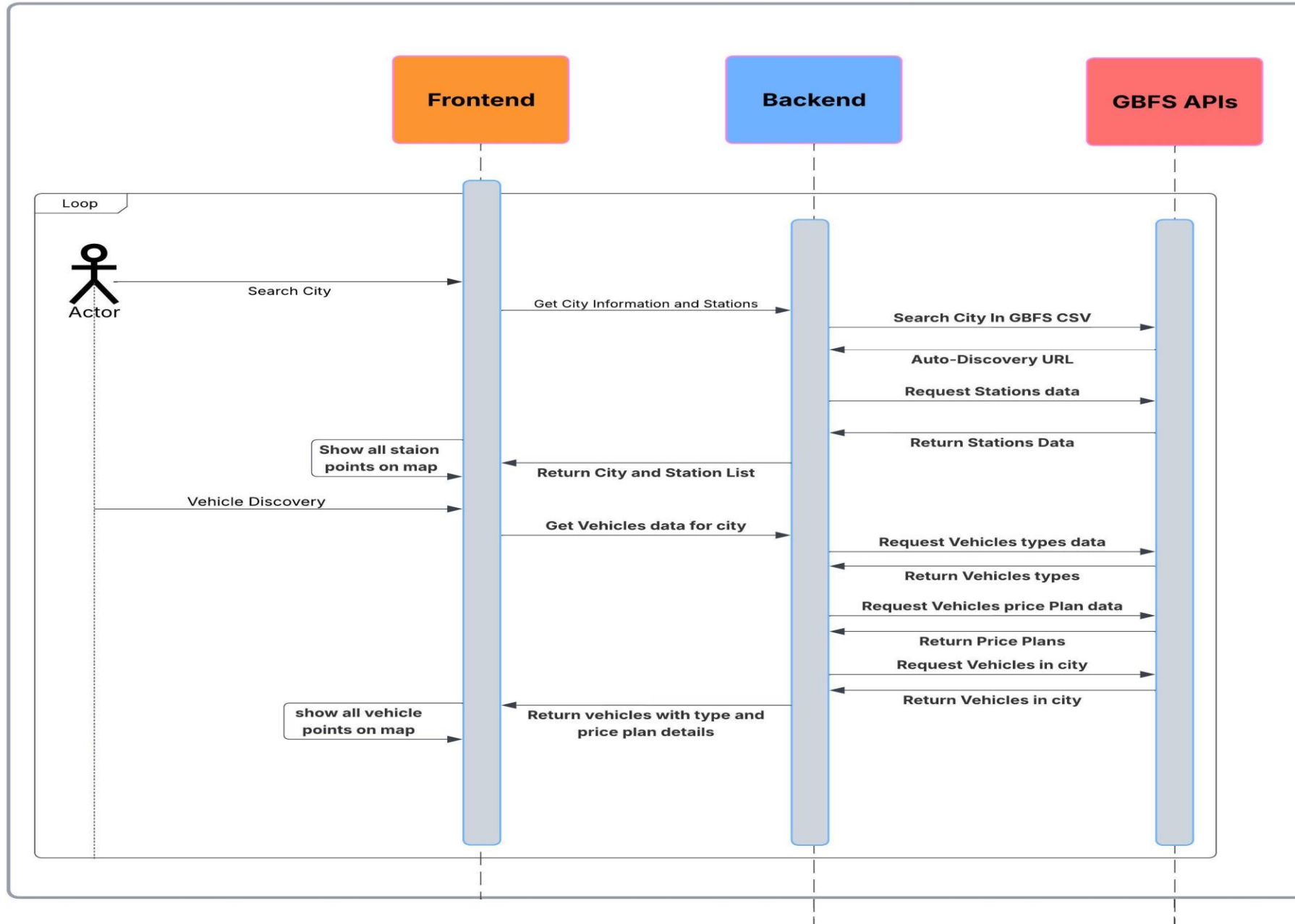
Back End Tech Stack



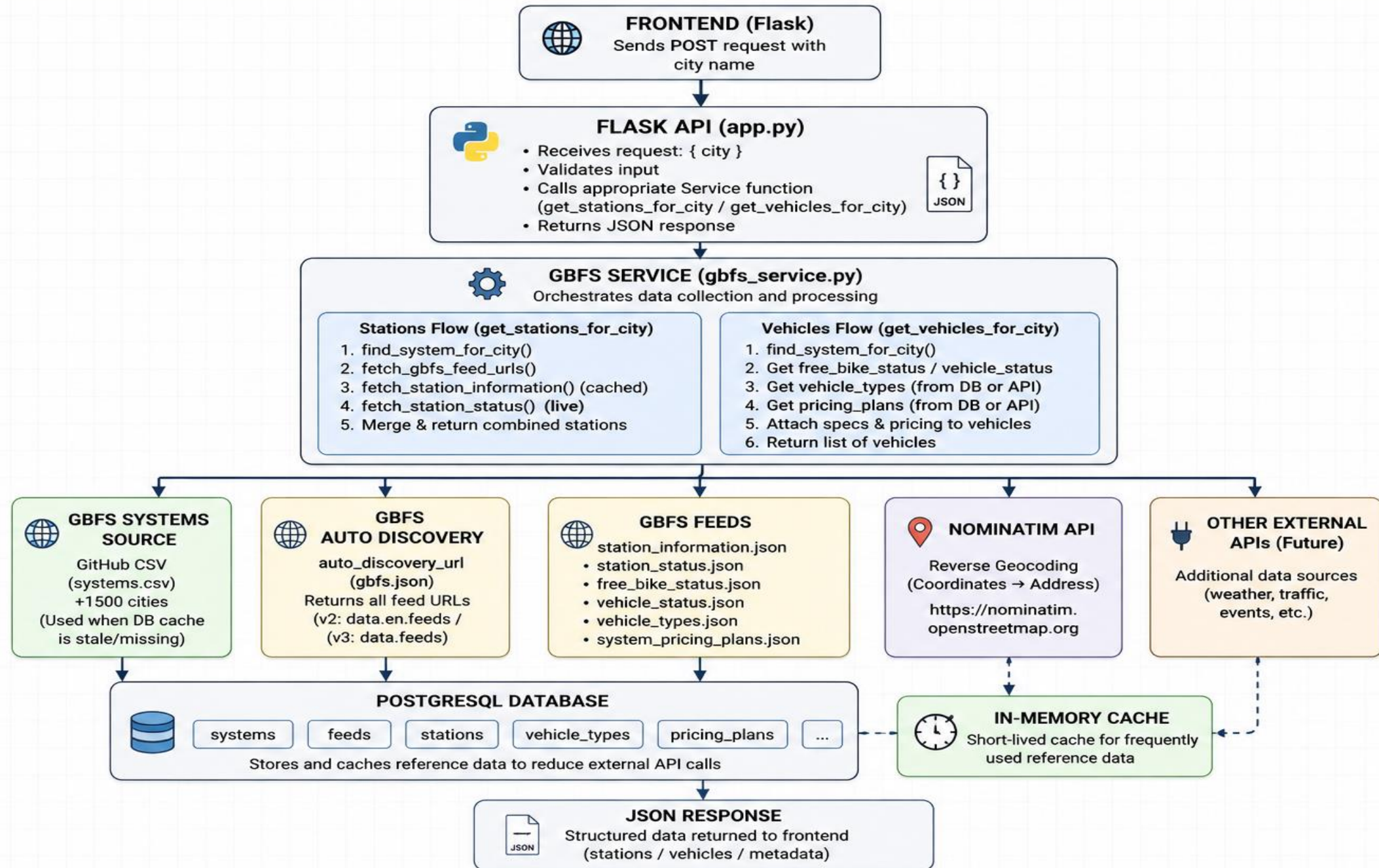
- **Flask:** Backend Services for GBFS and Nominatum API calling, DBOperations
- **GBFS** : General Bikeshare Feed Specification APIs Repository
- **Nominatim API:** Open-source geocoding with OpenStreetMap data API
- **Requests python Library:** To fetch data from GBFS API endpoints.
- **SQLite:** Saving snapshots of data fetched from APIs to avoid repeated Public API calls



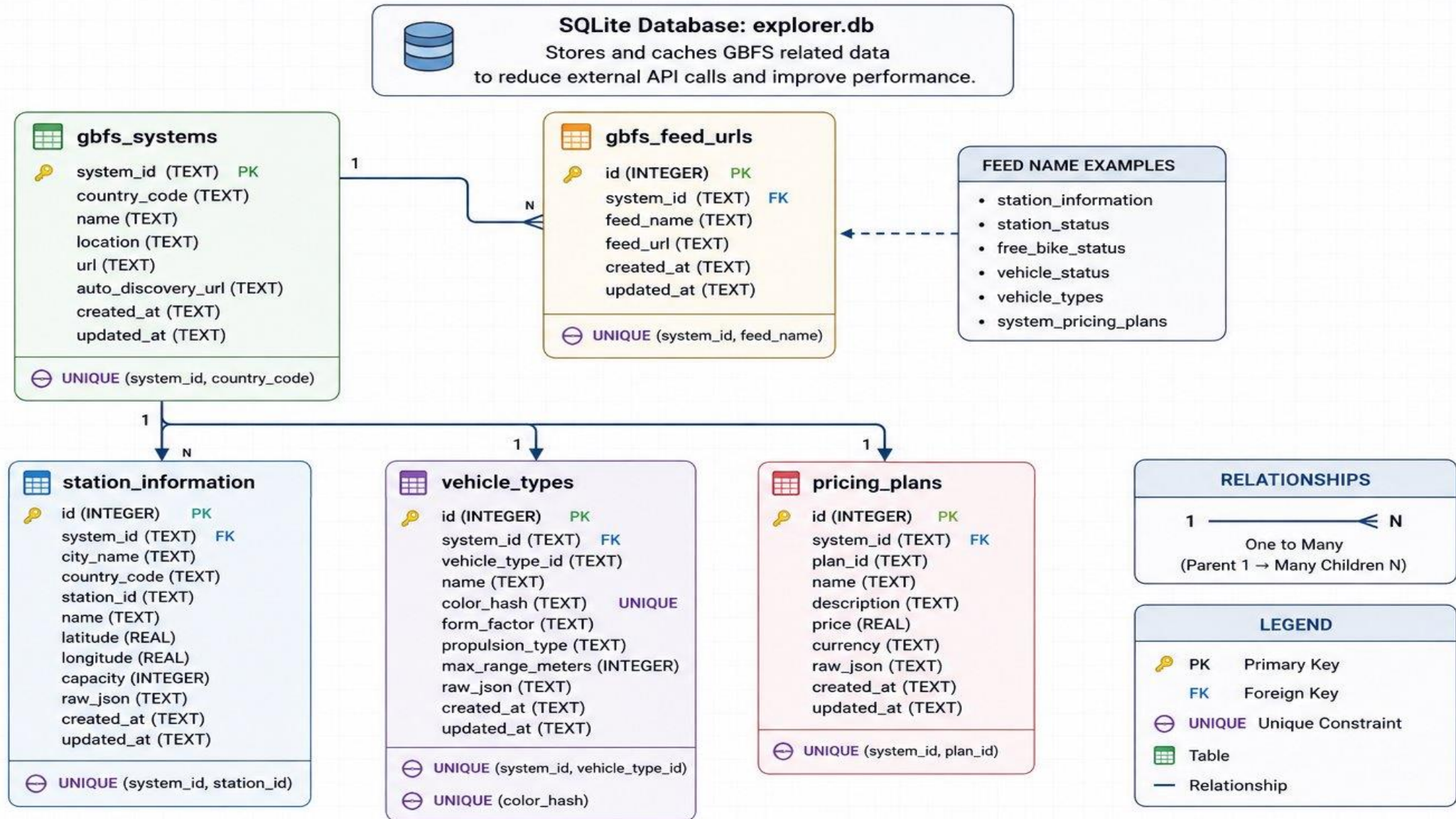
Frontend-Backend Sequence diagram



BACKEND ARCHITECTURE FLOW CHART



DATABASE DESIGN (ER DIAGRAM)



NOTES

- gbfs_systems is the parent table for all other tables.
- Each GBFS system can have multiple feeds, stations, vehicle types, and pricing plans.
- Feed URLs are stored per system and per feed type.
- Stations, vehicle types, and pricing plans are cached to minimize external API calls.

Backend Challenges

1. Inconsistent GBFS Response Structure

- One major challenge was that GBFS data is not fully consistent across all mobility providers.
- Different companies sometimes send different keys, missing fields, or slightly different response structures.

2. Handling Public API Usage Limits

- The project depends on public APIs such as GBFS feeds and Nominatim.
- When a city is searched for the first time, the backend stores a snapshot of reusable data such as GBFS systems, feed URLs, station information, vehicle types, and pricing plans.

3. Separating Live Data and Cacheable Data

- Not all data can be cached in the same way.
- live availability data like available bikes, available docks, or current vehicle locations changes frequently.
 - static or semi-static data → stored in DB
- live availability data → fetched fresh from GBFS APIs

4. Multiple Mobility Systems in One City

- Some cities can have more than one GBFS provider.
- This required careful handling of multiple systems, duplicate data, and combined.

5. Performance Optimization

- To improve performance, the backend uses a cache-first approach. It checks the database first and only calls external APIs when data is missing or expired.
- This helped reduce response time and made repeated searches for the same city much faster.

```
🌐 DB Write op save_gbfs_systems 🌐  
💿 DB read op search_city_gbfs 💿  
💿 DB read op search_city_gbfs 💿  
💿 DB read op get_feed_urls 💿  
🌐 DB Write op save_feed_urls 🌐  
🌐 DB Write op save_station_information bolt_stuttgart 🌐  
💿 DB read op get_feed_urls 💿
```

Backend Test Cases

1) City Search with New City

What was tested:

- Search a city that is not already stored in the database.

Steps:

1. User searches city, for example Stuttgart.
2. Backend checks DB for cached GBFS systems.
3. If not available, backend calls GBFS systems CSV/public API.
4. Backend stores matching system data in DB.
5. Backend returns city/station/vehicle data.

Actual Result:

- Backend successfully fetches data from GBFS API, stores reusable data in DB, and returns correct response to frontend.

Result: ✅ **PASS**


```
📀 DB read op search_city_gbfs 📀
📀 DB read op search_city_gbfs 📀
📀 DB read op search_city_gbfs 📀
📀 DB read op get_feed_urls 📀
📀 DB Read op get_station_information bolt_stuttgart 📀
📀 DB read op get_feed_urls 📀
📀 DB Read op get_station_information dott-stuttgart 📀
```

2)City Search with Cached Data

What was tested:

- Search the same city again after it was already searched once.

Steps:

1. User searches the same city again.
2. Backend checks DB first.
3. Backend finds cached system/feed/station data.
4. Backend avoids unnecessary public API calls.
5. Only live data like station status or vehicle availability is refreshed.

Actual Result:

- Backend uses cached data and reduces external API usage while still returning updated availability data.

Result:  **PASS**

```
🕒 DB read op search_city_gbfs 🕒
🕒 DB read op get_feed_urls 🕒
🕒 DB Read op get_station_information bolt_stuttgart 🕒
🕒 DB read op get_feed_urls 🕒
🕒 DB Read op get_station_information dott-stuttgart 🕒
🕒 DB read op get_feed_urls 🕒
GBFS feed discovery error: 404 Client Error: Not Found for url: https://
/api.mobidata-bw.de/sharing/gbfs/v3/lime_stuttgart/gbfs
🕒 DB read op get_feed_urls 🕒
🕒 DB read op get_feed_urls 🕒
🕒 DB Read op get_station_information regiorad_stuttgart 🕒
🕒 DB read op get_feed_urls 🕒
🕒 DB Read op get_station_information stadtmobil_stuttgart 🕒
```

3)GBFS Feed Discovery Error Handling

What was tested:

- One of the GBFS systems has an invalid or unavailable auto-discovery URL.

Steps:

1. User searches a city, for example Stuttgart.
2. Backend finds multiple GBFS systems listed for that city.
3. One provider's auto-discovery URL returns 404 Not Found.
4. Backend catches the error and skips that failed system.
5. Backend continues processing the remaining valid GBFS systems.

Actual Result:

- The backend should not crash if one GBFS provider fails. auto-discovery URL and continue fetching data from other available systems.

Result:  **PASS**

```
for vehicle in vehicles_list:
    type_id = vehicle.get("vehicle_type_id")
    plan_id = vehicle.get("pricing_plan_id")
    vehicle.update({
        "vehicle_type": vehicle_types.get(type_id),
        "price_plan": plansDict.get(plan_id),
    })
return vehicles, vehicle_type_names, vehicle_types, vehicle_colors
```

4) Merging vehicle Data with Type and Pricing Metadata

What was tested:

- Fetch vehicles for a city and attach vehicle type and pricing details.

Steps:

1. User opens vehicle map view.
2. Backend fetches vehicle availability data.
3. Backend checks if vehicle type and pricing plan exist in DB.
4. If missing, backend fetches metadata from GBFS feeds and saves it.
5. Backend attaches vehicle type and pricing details to each vehicle.

Actual Result:

- Frontend receives complete vehicle data including location, vehicle type, color, and pricing information.

Result:  PASS



Thank You

Questions and Answers

